

Build a Large Language Model from Scratch

This thesis is a summary of [1] and reports on the experience with it.

REPORT 4

Implementing a GPT Model

Giorgi Gogsadze

Revaz Goguadze

Supervised by Prof. Walter Tichy

Structure

1. Overview of GPT model architecture
2. Multi-Head Attention
3. Layer Normalization
4. Activation Functions
5. Feed-forward network
6. How Shortcut Connections Help
7. Transformer Block
8. Develop GPT Model and assess different configurations
9. Testing the GPT Model and Generating Text

Except for the exercise solutions and tests, all the code presented in this work are adapted from Chapter 4 of [1].

1. Overview of GPT model architecture

In this report, we present the implementation of a GPT model developed from scratch, including all architectural components required prior to the training phase.

Before starting the implementation, it is essential to understand the fundamental building blocks that create the model. Figure 1 provides a high-level overview of the GPT architecture that will be implemented and analyzed throughout this report.

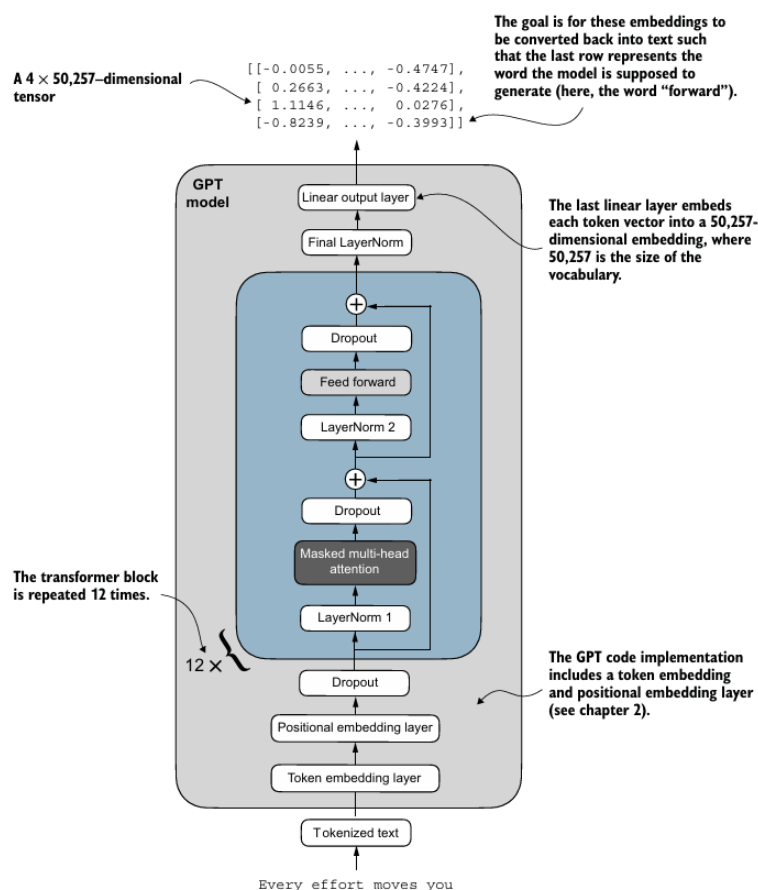


Fig.1: An overview of the GPT model architecture (Source: [1], p. 118)

At a high level, a GPT model processes an input sequence of tokens through several stages:

- First, **token embeddings** convert token indices into continuous vector representations that are combined with positional embeddings so the model also considers word order.
- These representations are then passed through a sequence of **transformer blocks** all sharing the same internal structure and two main subcomponents:
 - **Masked multi-head self-attention mechanism**
 - **Position-wise feed-forward network**

Both topics are further explained in this chapter.

- **Shortcut connections, layer normalization, and dropout** are applied throughout the model to stabilize training and improve generalization.
- After passing all transformer blocks, the final representations are projected through a **linear output layer**, producing one score for every possible word in the vocabulary (logits), representing the model's next token prediction.

GPT-2 Small model (124M parameters):

```
GPT_CONFIG_124M = {  
    "vocab_size": 50257,      # Vocabulary size  
    "context_length": 1024,   # Context length  
    "emb_dim": 768,           # Embedding dimension  
    "n_heads": 12,            # Number of attention heads  
    "n_layers": 12,           # Number of layers  
    "drop_rate_attn": 0.1,     # Dropout rate for multi-head attention  
    "drop_rate_shortcut": 0.1, # Dropout rate for shortcut connections  
    "drop_rate_emb": 0.1,     # Dropout rate for embedding layer  
    "qkv_bias": False         # Query-Key-Value bias  
}
```

2. Multi-Head Attention

Multi-Head Attention is the part of the model that allows each word to look at other words in the sentence and determine which are important. The input is first converted into queries, keys, and values, which are split across multiple attention heads so the model is capable of focusing on different relationships at the same time. A mask is used to make sure the model only looks at earlier words and not future ones. Finally, the information from all heads is combined, which produces **context vectors** for each token that are passed through a linear layer to produce the final output.

The full Python implementation of MultiHeadAttention class is given in the file “Chapter3” of [2].

3. Layer Normalization

Training deep neural networks like GPT can be challenging because, when information passes through many layers, the signal used to learn can become weaker and weaker. If the signal is too small, it is hard for the model to learn because it does not know how to adjust its weights correctly. This problem is addressed by **layer normalization** that maintains the values within each layer well-balanced, resulting in more stable learning and allowing the model for efficient training.

The core idea of layer normalization is to normalize the output of a neural network layer so that the values for each individual input have **a mean of zero and a variance of one**.

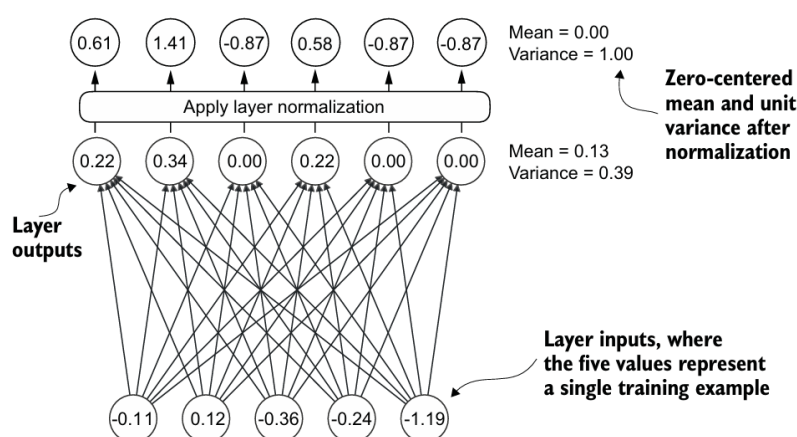


Fig.2: Layer normalization
(Source: [1], p. 100)

Figure 2 illustrates our goal. Normalization can be achieved by subtracting the mean from the input vector and dividing the result by the square root of the variance augmented with a small constant ϵ , which is included to prevent division by zero.

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

In GPT and other transformer models, layer normalization is applied at several points in each transformer block. The normalization is performed over the embedding values of each token, and the result is then adjusted using learnable **scaling** and **shifting** parameters so the model can choose the most useful value range. Because layer normalization works independently for each input and does not rely on batch statistics, it remains stable even with varying batch sizes, making it a key component for training large language models reliably.

4. Activation Functions

Activation functions add nonlinearity to neural networks, enabling them to model non-trivial patterns beyond simple linear transformations. In transformer-based architectures like GPT, the **Gaussian Error Linear Unit (GELU)** is often used.

The GELU function is defined as $GELU(x) = x \cdot \Phi(x)$

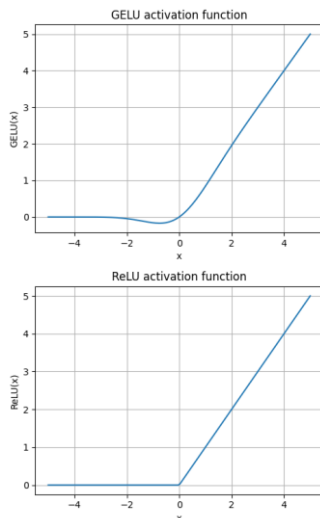
where $\Phi(x)$ is the cumulative distribution function of standard Gaussian distribution.

In practice, GPT-style models use the approximation that is more computationally efficient and still closely matches the exact GELU:

$$GELU(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} \cdot (x + 0.044715 \cdot x^3) \right] \right)$$

The **ReLU (Rectified Linear Unit)** activation used to be adopted widely, because of its simplicity and computational efficiency. However, GELU is a smoother activation function with superior optimization behavior during training and has been shown to enhance the performance of deep transformer architectures.

The comparison of both of these functions is shown below:



In the plot we can see that ReLU outputs a zero for negative inputs and is linear for positive values, while GELU is smooth nonlinear function which gives small outputs even for negative inputs. This smooth behavior has several advantages.

- Offers smooth transitions, which is good for optimization.
- Keeps neurons active on negative inputs, allowing more stable and richer gradients.

For these reasons, GELU has become a de-facto standard activation function in GPT and other large language models.

5. Feed-forward network

Besides the self-attention, each transformer block has a feed-forward network that processes each token representation independently. While attention combines information across tokens, the feed-forward network performs a non-linear transformations, refines and enriches these representations and improves the model's ability to capture complex patterns.

In GPT, the feed-forward network is implemented as a small neural network with **two linear layers and a GELU activation function in between**.

Figure 3 below shows how the first linear layer expands the embedding dimension by a factor of four, which allows the model to explore a richer set of feature interactions. Then the GELU activation introduces nonlinearity. Finally, the second linear layer projects the representation back to the original embedding dimension.

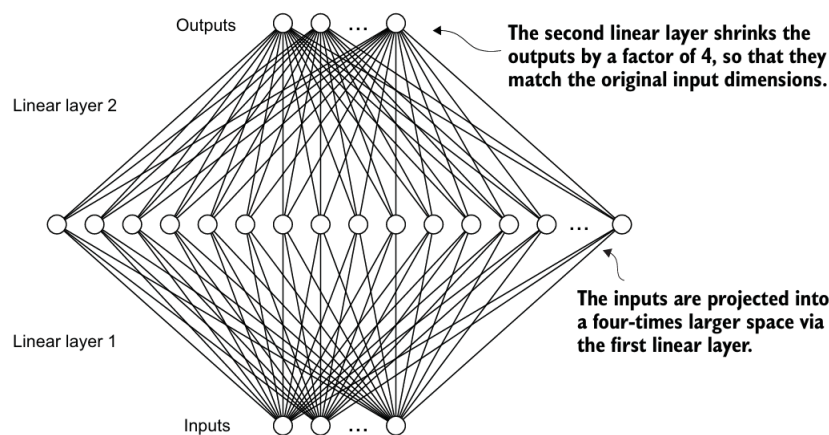


Fig.3: expansion and contraction of the layer
(Source: [1], p. 108)

By keeping the input and output dimensions the same, transformer blocks can be stacked. This allows the feed-forward network to increase the model's ability to learn complex patterns while preserving a simple and flexible structure.

6. How Shortcut Connections Help

Shortcut connections are used in deep neural networks to address the vanishing gradient problem. Instead of forcing information to pass only through a long sequence of transformations, shortcut connections add a layer's input directly to its output. This provides an easier path for both information and learning signals to flow through the network.

To illustrate the effect of shortcut connections, Raschka[1] constructs a deep neural network composed of multiple linear layers followed by GELU activations and compares two versions of the model: one with shortcut connections and one without. We also did a similar experiment that gave us the result:

```
Model without shortcut
layers.0.0.weight has gradient mean of 0.00011171719233971089
layers.1.0.weight has gradient mean of 0.0002804722171276808
layers.2.0.weight has gradient mean of 0.0004977626376785338
layers.3.0.weight has gradient mean of 0.006228148005902767
layers.4.0.weight has gradient mean of 0.015021126717329025
layers.5.0.weight has gradient mean of 0.056009769439697266

Model with shortcut
layers.0.0.weight has gradient mean of 0.002781738992780447
layers.1.0.weight has gradient mean of 0.0027199035976082087
layers.2.0.weight has gradient mean of 0.0002717698225751519
layers.3.0.weight has gradient mean of 0.001978363608941436
layers.4.0.weight has gradient mean of 0.005832917056977749
layers.5.0.weight has gradient mean of 0.04165162891149521
```

It's clear that in the model without shortcut the gradients become smaller as it progresses from the last layer (layers.5) to the first layer (layers.0), which is a phenomenon called the **vanishing gradient problem**. When gradients vanish, earlier layers receive very little learning signal, making it difficult for the network to update its parameters effectively.

In contrast, Model with shortcut demonstrates the gradient value stabilizes as we progress toward the first layer and doesn't shrink to a vanishingly small value. Therefore, shortcut connections enable effective learning across the entire network.

7. Transformer Block

The **transformer block** is the fundamental building unit of modern large language models. In the GPT-2 Small architecture, this block is repeated twelve times and combines several key components previously introduced: **masked multi-head attention, feed-forward networks, layer normalization, GELU activations, dropout, and shortcut connections**. Together, these components enable the model to capture both contextual relationships across tokens and complex nonlinear transformations within each token representation. Importantly, all operations preserve the fixed embedding dimension (768), enabling multiple transformer blocks to be stacked without changing the data shape.

Structure and Data Flow

The transformer block follows a **Pre-Layer Normalization** design, where layer normalization is applied *before* both the attention and feed-forward sublayers.

The forward pass consists of two main stages:

1. **Self-Attention Stage**

The input tensor is first normalized and then passed through the masked multi-head attention module. Dropout is used for regularization and a shortcut connection then adds the original input to the attention output.

2. **Feed-Forward Stage**

The result is again normalized and passed through the feed-forward network, which applies a nonlinear transformation independently to each token. After dropout, another shortcut connection adds the input of this stage back to the output.

Exercise 4.1: Parameter Comparison

To compare the computational cost of transformer components, we examine the number of parameters in the feed-forward and multi-head attention modules:

```
tfb = TransformerBlock(GPT_CONFIG_124M)
total_params_ff = sum(p.numel() for p in tfb.ff.parameters())
total_params_att = sum(p.numel() for p in tfb.att.parameters())
```

The resulting parameter counts are:

- **Feed-forward module:** 4,722,432 parameters
- **Multi-head attention module:** 2,360,064 parameters

Thus, the feed-forward network contains approximately twice as many parameters as the attention module. This difference exists because, as shown before, the feed-forward network temporarily makes each token representation much larger before shrinking it back to its original size.

8. Develop GPT Model and assess different configurations

In the previous sections, we implemented and analyzed the individual building blocks of a GPT architecture. We now put together these components to produce a fully functional LLM model. The full GPT model is implemented as follows:

```
class GPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate_emb"])
        # Stacked transformer layers
        self.trf_blocks = nn.Sequential(
            *[TransformerBlock(cfg) for _ in range(cfg["n_layers"])]
        )
        self.final_norm = LayerNorm(cfg["emb_dim"])
        # Project hidden states back to vocabulary size
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"], bias=False
        )

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeddings = self.tok_emb(in_idx)
        pos_embeddings = self.pos_emb(
            torch.arange(seq_len, device=in_idx.device)
        )
        x = tok_embeddings + pos_embeddings
        x = self.drop_emb(x)
        x = self.trf_blocks(x) # Core processing
        x = self.final_norm(x)
        # Output unnormalized probabilities (logits)
        logits = self.out_head(x)
        return logits
```

First, the forward pass maps token indices into embeddings, adds positional information, and feeds the resulting tensor to the stacked transformer blocks. After final normalization, the output is projected into the vocabulary space, producing logits that represent unnormalized probabilities for the next token in the sequence. For an input of shape [batch_size, num_tokens], the output has shape [batch_size, num_tokens, vocab_size].

Exercise 4.2: Assessing Different GPT Configurations

Using the same GPTModel class and modifying only the configuration dictionary with values shown in the table below, we instantiate four GPT-2 variants. This demonstrates the scalability of the architecture:

Model	Embedding Dimension	Number of Layers	Number of Attention Heads	Trainable Parameters (with weight tying)	Total Parameters	Approx. Size
GPT-2 Small	768	12	12	124,412,160	163,009,536	621.83 MB
GPT-2 Medium	1024	24	16	354,749,440	406,212,608	1549.58 MB
GPT-2 Large	1280	36	20	773,891,840	838,220,800	3197.56 MB
GPT-2 XL	1600	48	25	1,557,380,800	1,637,792,000	6247.68 MB

The results show that increasing the embedding dimension, number of attention heads, and number of transformer layers rapidly increase both parameter count and memory usage, highlighting the computational and storage demands of large language models.

9. Testing the GPT Model and Generating Text

After implementing the GPT architecture, we do basic tests to confirm that the model correctly processes tokenized inputs, produces outputs with expected shape, and supports auto-regressive text generation.

Tokenization and Model Output

We first encode two short input sentences using the GPT-2 tokenizer to construct a batch of token IDs:

```
tokenizer = tiktoken.get_encoding("gpt2")
batch = []
txt1 = "The Education is important"
txt2 = "People together can do"
batch.append(torch.tensor(tokenizer.encode(txt1)))
batch.append(torch.tensor(tokenizer.encode(txt2)))
batch = torch.stack(batch, dim=0)
```

The resulting batch has shape [2, 4], which is two input sequences with four tokens each. The shape of the output tensor of the GPT model when you pass it through is:

```

torch.manual_seed(123)
model = GPTModel(GPT_CONFIG_124M)
out = model(batch)
print("\nOutput shape:", out.shape)

```

RESULT: Output shape: torch.Size([2, 4, 50257])

Each vector in the last dimension contains the unnormalized scores (logits) of the model for every possible next tokens in the vocabulary.

Text Generation

When generating text, the model generates one token at a time, based on previous tokens. This is called **autoregressive generation**.

At each step:

1. Input to the model is the current sequence.
2. It predicts the scores (logits) for all the possible next tokens.
3. The softmax function takes these scores and converts them into probabilities.
4. Argmax function is used to get the most likely token and add it to the sequence.

This method is called **greedy decoding** because it always chooses the most likely next token.

```

def generate_text_simple(model, idx, max_new_tokens, context_size):
    for _ in range(max_new_tokens):
        # Crop current context if it exceeds the model's maximum supported-length
        idx_cond = idx[:, -context_size:]
        with torch.no_grad():
            logits = model(idx_cond)
        # Focus only on the last time step (the next token to be predicted)
        logits = logits[:, -1, :]
        probas = torch.softmax(logits, dim=-1)
        #Greedy decoding
        idx_next = torch.argmax(probas, dim=-1, keepdim=True)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx

```

After applying softmax, we get a vector of size 50257. Each value represents the probability of a token with index i , where $i \in [0, 50256]$

We use greedy decoding, therefore we take the index with the highest probability. That is what the **argmax** function returns.

This process is done for a fixed number of steps, gradually increasing the length of the input sequence and generating new tokens one at a time. The generated token IDs are then converted back to human-readable text using the tokenizer's decode method.

Generation Results and Interpretation

Now we apply the generation function to the sample inputs:

```
model.eval()
out = generate_text_simple(model=model, idx = batch, max_new_tokens = 7,
context_size=GPT_CONFIG_124M["context_length"])
for idx in out:
    decoded_text = tokenizer.decode(idx.tolist())
    print(decoded_text)
```

RESULT:

The Education is important criminal?) curriculumhern ic fres Ic

People together can doftime intensive💎 REALLY tumble Nay habitats

While the generated text is syntactically fragmented and largely incoherent, this behavior is expected. The model has **not been trained yet**.

These tests demonstrate that the implemented GPT model:

- correctly processes tokenized input,
- produces outputs with the expected dimensionality.
- supports iterative next-token generation.

Meaningful and coherent text generation requires training the model on large text corpora, which will be addressed in the subsequent chapter.

Exercise 4.3: Using separate dropout parameters

Instead of using a single global dropout rate, we define separate dropout parameters for different parts of the model: the embedding layer, the shortcut connections, and the multi-head attention module. This is done by extending the configuration with three values: `drop_rate_emb`, `drop_rate_shortcut`, and `drop_rate_attn`.

These values are then applied in the model:

- **drop_rate_emb** is used in the embedding layer
- **drop_rate_shortcut** is used in the residual (shortcut) connections inside each Transformer block
- **drop_rate_attn** is input to the multi-head attention module

This decoupling enables more granular regularization control, leading to greater flexibility and potentially performance of models.

References

[1] Raschka, S. (2024). *Build a large language model (from scratch)*. Shelter Island, NY: Manning Publications.

[2] Giorgi Gogsadze, Revaz Goguadze. LLM from Scratch Code Repository. Available at: <https://github.com/GiorgiGogsadze/LLM-From-Scratch-Thesis>